

DSA STUDY BLUEPRINT SERIES

C++ Vectors Blueprint

Dynamic Arrays and STL Container Architecture

Section 02 • C++ Core Data Structures & Algorithms

Why Vectors? The Dynamic Array Solution

Overcoming the limitations of static array sizes

Static Arrays (Fixed Size)

- Size must be declared at compile time.
- Cannot expand or shrink at runtime.
- Risk of stack overflow for large arrays.
- Stored in contiguous stack space.

C++ Vectors (Dynamic Size)

- Automatically reallocates memory as needed.
- Resides on the Heap (no stack overflow risk).
- Rich STL interface (`push_back` , `pop_back`).
- Safe element access with boundary checks (`.at()`).

Size vs. Capacity & Memory Doubling

Understanding vector's underlying scaling mechanism

The Scaling Scheme

When the vector exceeds its current **Capacity**:

- 1. A new memory block with **double** capacity is allocated.
- 2. Existing elements are copied to the new block.
- 3. The old memory block is deallocated.

Capacity Growth Progression:

0 → 1 → 2 → 4 → 8 → 16 → 32 → 64 ...

Memory State Tracking

Action	Size	Capacity	Internal Array
Initial State	0	0	[]
push_back(10)	1	1	[10]
push_back(20)	2	2	[10, 20]
push_back(30)	3	4	[10, 20, 30, _]
push_back(40)	4	4	[10, 20, 30, 40]
push_back(50)	5	8	[10..50, _, _, _]

Vector Operations in Code

Common operations and dynamic input patterns

```
#include
#include
using namespace std;

int main() {
    vector vec = {10, 20, 30}; // Initial list
    vec.push_back(40);          // Insert element at the back
    vec.pop_back();             // Removes 40

    // Size vs Capacity
    cout << "Size: " << vec.size() << " | Capacity: " << vec.capacity() << endl;

    // Iterating elements (safe access)
    for (int i = 0; i < vec.size(); i++) {
        cout << vec.at(i) << " "; // Throws out_of_range if i is invalid
    }
    return 0;
}
```

Complexities & Best Practices

Writing efficient vector code in production

Performance Characteristics

Operation	Time Complexity
Access (<code>v[i]</code> , <code>v.at(i)</code>)	$O(1)$
Insert/Remove at end	$O(1)$ amortized
Insert/Remove at start	$O(N)$ (element shifts)

Essential Guidelines

- **Pass by Reference:** Always pass vectors as `void func(vector& v)` to avoid costly copy overhead.
- **Use `reserve()`:** If final size is known, use `v.reserve(N)` to avoid multiple reallocations.
- **Bounds Check:** Prefer `v.at(i)` over `v[i]` for critical debugging.